OTUS · ИИ-архитектор

Домашнее задание №5

Низкоуровневое проектирование (LLD)

Knowledge Base Filling Agent

C3 · OpenAPI 3.1 · Sequence · BPMN · DFD

Доказательный конвейер Knowledge Base Filling Agent

Автоматический компонент не имеет права выполнять переход CLAIM → FACT

Архитектурный принцип

SOURCE → CAPTURE → CHUNK → CLAIM / CANDIDATE → HUMAN REVIEW → VERIFIED KNOWLEDGE

Репозиторий: github.com/VictorKVS/OTUS-

Папка сдачи: ДЗ_05_OSINT_KB_Agent_LLD

PR #12 · merge commit d39000c866...

1. Условие и результат

Цель занятия: детализировать высокоуровневый контейнер до компонентов, спроектировать API и показать последовательность взаимодействий. Для практической части выбран контейнер Knowledge Base Filling Agent.

Требование	Артефакт
LLD одного контейнера	Knowledge Base Filling Agent
C3 Component Diagram	architecture/C3_KB_AGENT_COMPONENTS.svg
OpenAPI	api/openapi.yaml · OpenAPI 3.1
Sequence Diagram	architecture/SEQUENCE_EVIDENCE_TO_KB.md
Дополнительные потоки	BPMN + DFD

Что делает агент

Агент принимает доказательственный пакет от OSINT/Screening-контура, проверяет целостность и допустимость, создаёт устойчивые chunks, извлекает кандидатов сущностей/утверждений/связей, сохраняет provenance, выявляет противоречия и направляет спорные объекты на человеческую проверку.

Критическая архитектурная граница: автоматический компонент не имеет endpoint и полномочия для прямого CLAIM → FACT.

2. C3 Component Diagram

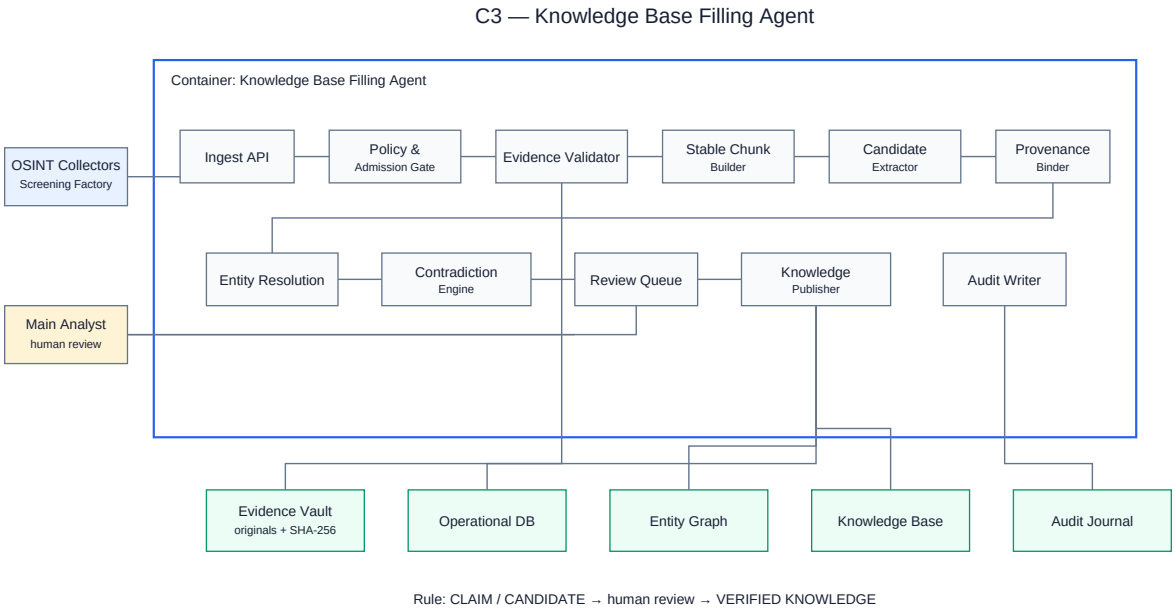


Диаграмма показывает внутренние компоненты контейнера и связи с OSINT Collectors, Main Analyst и хранилищами. Evidence Vault является доказательственным слоем; Knowledge Base получает только утверждённые знания.

3. Ответственность компонентов

Компонент	Ответственность
Ingest API	Принимает EvidencePackage и обеспечивает idempotency
Policy & Admission Gate	Проверяет purpose, legal basis и access class
Evidence Validator	Проверяет manifest, lineage, hash и сохраняет оригинал
Stable Chunk Builder	Создаёт воспроизводимые chunks с локаторами
Candidate Extractor	ENTITY / EVENT / CLAIM / DEFINITION / RELATION
Provenance Binder	Связывает candidate с source/capture/chunk/tool version
Entity Resolution	Exact merge; fuzzy merge требует review
Contradiction Engine	Конфликты, supersession, stale и research gaps
Review Queue	Human gate: APPROVE / REWORK / REJECT
Knowledge Publisher	Versioned write в KB и проекция в graph
Audit Writer	Append-only audit trail

Принцип проектирования

Компоненты разделены по ответственности: admission, evidence integrity, extraction, provenance, entity resolution, contradiction analysis, review и publication. Это позволяет отдельно тестировать каждый этап и не смешивать добычу материала с утверждением факта.

4. OpenAPI 3.1

Контракт API оформлен в api/openapi.yaml. Основные операции:

Метод	Endpoint	Назначение
POST	/evidence-packages	Регистрация evidence package
GET	/evidence-packages/{packageId}	Статус обработки
POST	/evidence-packages/{packageId}/process	Запуск deterministic pipeline
GET	/review-items	Очередь human review
POST	/review-items/{id}/decision	Решение APPROVE / REWORK / REJECT
GET	/cases/{caseId}/coverage	Coverage и research gaps
GET	/cases/{caseId}/lineage/{objectId}	Трассировка до capture/chunk

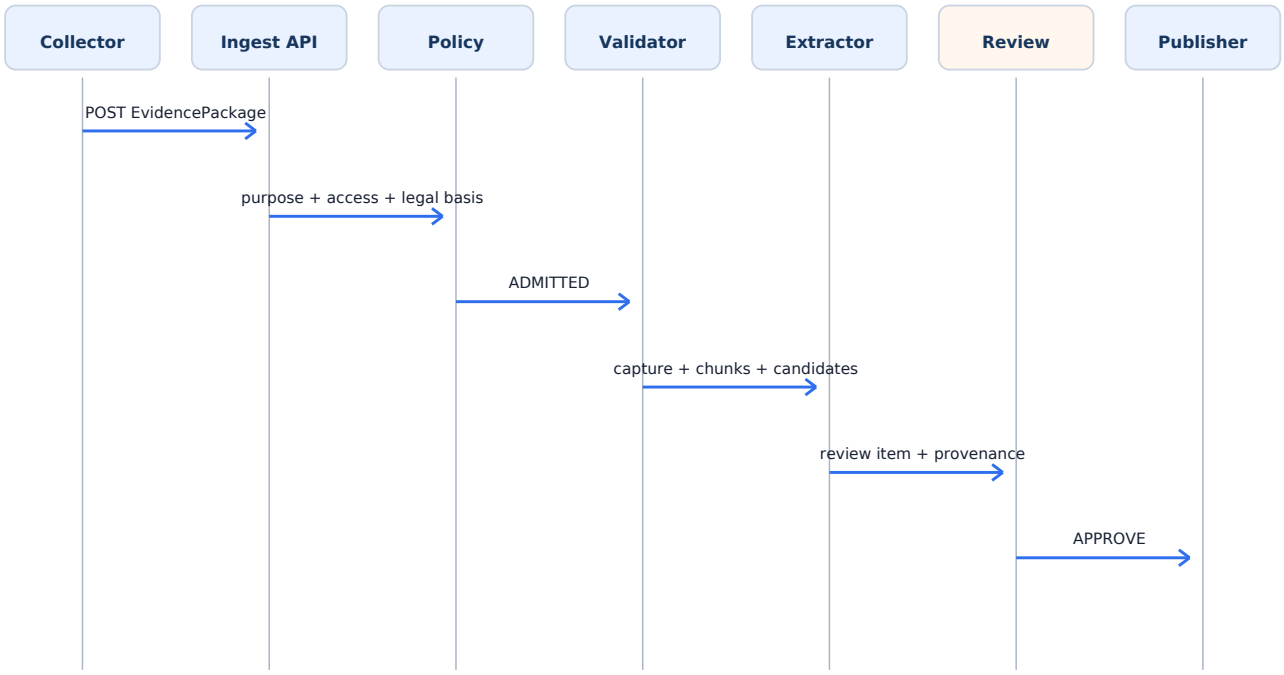
Ключевые ограничения

• Bearer authentication для внутреннего API.
• 403 при нарушении policy/access class.
• 409 для idempotency/state conflicts.
• Отдельный human-review endpoint.
• Отдельный lineage endpoint для трассировки производного объекта до source capture и stable chunks.

OpenAPI намеренно не содержит операции, которая позволяет автоматической модели присвоить объекту статус FACT.

5. Sequence Diagram

Ключевой сценарий: Evidence Package → Human Review → Knowledge Base



REWORK и REJECT фиксируются в Audit Journal и не публикуются как FACT

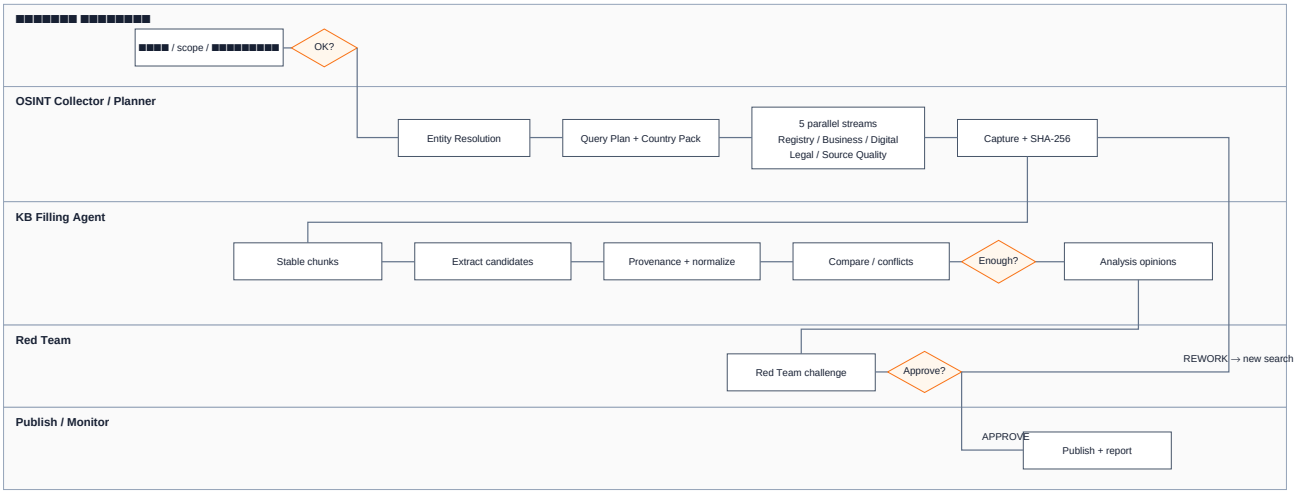
Ключевой сценарий

Collector регистрирует EvidencePackage. Policy Gate проверяет основание и класс доступа. Validator фиксирует оригинал и SHA-256. Extractor формирует кандидатов. Review Queue передаёт их аналитику. Только решение APPROVE позволяет Knowledge Publisher выполнить versioned write в Knowledge Base и Entity Graph; REWORK/REJECT остаются в Audit Journal.

Исходная Mermaid sequence diagram

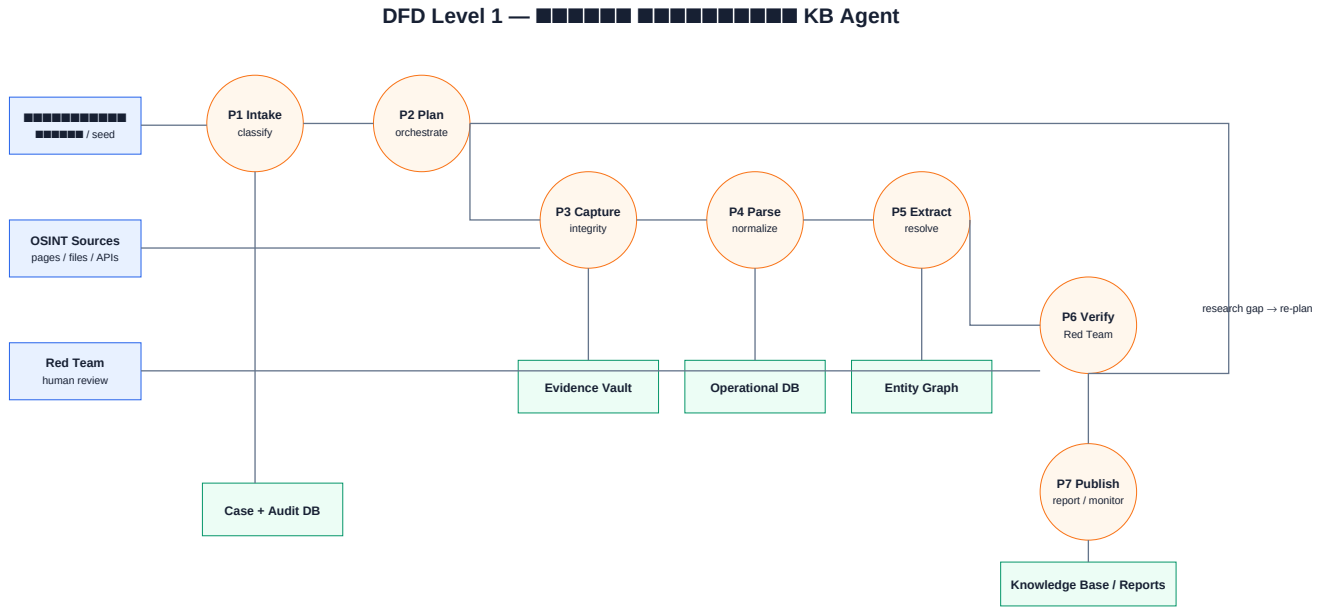
6. BPMN — бизнес-процесс

BPMN — [redacted]



BPMN дополняет LLD процессным взглядом: admission, параллельный сбор, нормализация, проверка достаточности, Research Gap, Red Team, Human Review и публикация. Возврат REWORK формирует управляемый цикл доисследования вместо заполнения пробелов предположениями.

7. DFD — потоки информации



DFD показывает физическое движение данных между источниками, агентом и хранилищами. Evidence Vault хранит оригиналы и hashes; Operational DB — состояние процесса; Entity Graph — производную сеть связей; Knowledge Base — проверенный экспертный слой; Audit Journal — неизменяемый след действий.

8. Информационная модель и доказательность

Доказательный конвейер Knowledge Base Filling Agent

Автоматический компонент не имеет права выполнять переход CLAIM → FACT

Разделение объектов

Класс	Назначение
SOURCE	источник как объект происхождения
SOURCE_CAPTURE	сохранённая копия + URL + UTC + SHA-256
STABLE_CHUNK	воспроизводимый фрагмент материала
CLAIM	утверждение источника
FACT	объект, утверждённый после review
INFERENCE	аналитический вывод из фактов
HYPOTHESIS	версия, требующая проверки
RELATION	типизированная связь с evidence refs
REVIEW	APPROVE / REWORK / REJECT
AUDIT_EVENT	кто, когда и что изменил

9. Адреса, проверка и сдача

Репозиторий	Папка ДЗ	PR #12

GitHub-адреса

Репозиторий VictorKVS/OTUS-

Папка ДЗ_05_OSINT_KB_Agent_LLD

README — точка входа для преподавателя

PDF для сдачи

Pull Request #12

```
Merge commit: d39000c866c9b51b19801f30a95a5ea16d37605c
```

Что отправить преподавателю

1. Ссылку на папку ДЗ.
2. PDF DZ05_LLD_OSINT_KB_AGENT_SUBMISSION.pdf.
3. При необходимости — прямую ссылку на OpenAPI и C3.

Комплект самодостаточен: README → C3 → OpenAPI → Sequence → BPMN/DFD → исходники.